

Expert Background: Tile Engine, Dispatcher, the Wrong PR, and How to Fix It

Complete technical reference — architecture, code, design decisions, failure analysis, and correct path forward.

Chapter 1: The GPU Hardware Context

What a GPU compute unit actually does

An AMD GPU like the MI300X contains 304 Compute Units (CUs). Each CU has 64 threads that execute together in lockstep — AMD calls this group a **wavefront** (NVIDIA calls it a warp). Every thread in a wavefront executes the same instruction at the same time on different data. This is the Single Instruction Multiple Data (SIMD) model.

The hardware instruction that matters most for matrix multiply is **MFMA** — Matrix Fused Multiply-Add. One MFMA instruction takes two small matrices (e.g. 32×32 and 32×16) and accumulates their product into a 32×32 result register block — all in one clock cycle per wavefront. MFMA is the reason GPUs are fast at GEMM.

The key MFMA shapes available on gfx942 (MI300X) for fp16:

```
32x32x8  - 32-row x 32-col output, reads 8 columns of A and 8 rows of B per call
32x32x16 - wider K accumulation
16x16x16 - smaller output tile
16x16x32 - wider K
4x64x16  - very wide N, narrow M
64x4x16  - very wide M, narrow N
```

These are hardware constraints, not software choices. The kernel must use exactly one of these shapes, and the tile dimensions chosen for the kernel must be exact multiples of the MFMA shape.

Memory hierarchy and why it matters

GPU DRAM (HBM3 on MI300X)	~3.2 TB/s bandwidth	slow but huge
■■■ L2 Cache (per die)	~15 TB/s bandwidth	
■■■ L1 Cache / LDS	~100 TB/s bandwidth	fast but tiny (64 KB per CU)
■■■ Registers	effectively infinite BW	fastest, 512 per thread

A GEMM kernel's performance is dominated by how cleverly it moves data through this hierarchy. The tile sizes in a kernel config (tile_m, tile_n, tile_k) determine how large a chunk each block loads into LDS (shared memory) at once. Too small: arithmetic intensity is low, memory bottleneck. Too large: LDS overflow, kernel launch failure.

The tiling hierarchy

A GEMM on a $4096 \times 4096 \times 4096$ matrix is split into a three-level tile hierarchy:

The divisibility constraint — `tile_m % (warp_m × warp_tile_m) == 0` — is a hard hardware requirement. Violating it produces either a compile error or wrong answers.

Chapter 2: Tile Engine — Complete Architecture

What Tile Engine is

Tile Engine is AMD's original system for generating, compiling, and running optimized GEMM kernels. It treats each (dtype, layout, pipeline, tile config, GPU arch) combination as an independent artifact: one config $\rightarrow$ one header $\rightarrow$ one compiled binary.

The complete Tile Engine pipeline

```

■ Tile Engine JSON Config File ■
■ {
■   "datatype": "fp16",
■   "layout": "rcr",
■   "tile_config": {
■     "tile_m": {"values": [256]},
■     "tile_n": {"values": [128]},
■     "tile_k": {"values": [32]},
■     "warp_m": {"values": [4]},
■     "warp_n": {"values": [1]},
■     "warp_tile_m": {"values": [32]},
■   }
■ }

```

```

    },
    "trait_config": {
        "pipeline":{"values":["compv3"]},
        "scheduler":{"values":["intrawave"]},
        ...
    }
}
}

unified_gemm_codegen.py (Python)
• Expands Cartesian product of all {"values":[...]} lists
• Validates each combo (tile divisibility, warp shape whitelist)
• Renders C++ Jinja2 templates for each valid combo

Generated C++ Header
gemm_fp16_rcr_compv3_default_
intrawave_False_False_False_
False_256x128x32_4x1x1_
32x32x16.hpp
#define SelectedKernel ...
#define ADataType ck_tile::half_t
#define KERNEL_NAME "..."

hipcc + ck_tile headers

Compiled Binary Executable
benchmark_gemm_universal_
fp16_rcr_compv3_default_
intrawave_False_False_False_
False_256x128x32_4x1x1_
32x32x16

./benchmark_gemm_universal... -m=1024 -n=1024 -k=1024 -verify=1

Output:
PASSED
Time: 11.8 ms (84.7 GFLOP/s)

```

What the binary name encodes

The binary name is not arbitrary — it is a full serialization of the kernel config:

```

benchmark_gemm_universal_
fp16_      ← dtype_a (and dtype_b, dtype_c)
rcr_       ← layout: A=row, B=col, C=row
compv3_    ← pipeline
default_   ← epilogue
intrawave_ ← scheduler (raw TE string, "default" would map to "auto" in dispatcher)
False_     ← pad_m
False_     ← pad_n

```

This name is also the **kernel registry key** inside Tile Engine's own lookup system.

What unified_gemm_codegen.py actually is

This is the most misunderstood file in the project. It lives at:

dispatcher/codegen/unified_gemm_codegen.py

It belongs to the Dispatcher, not to Tile Engine.

TE used it as an external tool. The Dispatcher owns it and calls it internally via CMake. When the current PR's `drive_codegen.py` invokes it as a subprocess, it is treating the Dispatcher's own internal tool as if it were a TE artifact — a fundamental category error.

Tile Engine's structural problems

Problem	Description
Build explosion	500 configs = 500 hipcc compilations; full CI takes hours
No runtime selection	Kernel choice is fixed at compile time by binary name
No shared code	Each binary is a complete standalone program; no sharing
Hard to scale	O(N) binaries per N configs; adding a new GPU triples the count
No Python interface	Only CLI invocation; no library API for programmatic use
Duplication	The kernel logic is templated C++; 500 binaries differ only in template args

Chapter 3: The Dispatcher — Complete Architecture

The three-layer architecture

```

■ LAYER 3: Python Interface
■ dispatcher/python/ctypes_utils.py
■
■ from ctypes_utils import KernelConfig, Registry, detect_gpu_arch
■
■ arch = detect_gpu_arch()           # queries rocm_info → "gfx942"
■ reg  = Registry()                  # creates in-process registry
■ cfg  = KernelConfig()              # describes the kernel you want

```

[illegible]

KernelKey — The identity card (kernel_key.hpp)

Every kernel in the Dispatcher is described by a `kernelkey` struct. It has two sections:

Signature — describes WHAT mathematical operation is computed. Two kernels with the same Signature produce the same mathematical result (given the same inputs).

Algorithm — describes HOW it is computed. Two kernels with the same Signature but different Algorithm produce the same result with different performance.

```

struct Algorithm {
    struct TileShape    { uint16_t m, n, k; } tile_shape;    // block tile
    struct WaveShape    { uint8_t  m, n, k; } wave_shape;    // waves per block
    struct WarpTileShape{ uint8_t  m, n, k; } warp_tile_shape; // MFMA shape
    Pipeline    pipeline;
    Scheduler    scheduler;
    Epilogue    epilogue;
    uint16_t    block_size;
    bool        double_buffer, persistent, preshuffle, transpose_c;
    uint8_t     num_wave_groups;
    bool        pad_m, pad_n, pad_k;
} algorithm;

std::string gfx_arch;

std::string encode_identifier() const; // the registry key
};

```

encode_identifier() — The registry key (verbatim from kernel_key.hpp)

```

inline std::string KernelKey::encode_identifier() const {
    std::ostringstream oss;
    oss << to_string(signature.dtype_a) << "_";
    oss << to_string(signature.layout_a)
        << to_string(signature.layout_b)
        << to_string(signature.layout_c) << "_";
    oss << to_string(algorithm.pipeline) << "_";
    oss << to_string(algorithm.epilogue) << "_";
    oss << to_string(algorithm.scheduler) << "_";
    oss << (algorithm.pad_m ? "True":"False") << "_";
    oss << (algorithm.pad_n ? "True":"False") << "_";
    oss << (algorithm.pad_k ? "True":"False") << "_";
    oss << (algorithm.persistent ? "True":"False") << "_";
    oss << algorithm.tile_shape.m << "x"
        << algorithm.tile_shape.n << "x"
        << algorithm.tile_shape.k << "_";
    oss << unsigned(algorithm.wave_shape.m) << "x"
        << unsigned(algorithm.wave_shape.n) << "x"
        << unsigned(algorithm.wave_shape.k) << "_";
    oss << unsigned(algorithm.warp_tile_shape.m) << "x"
        << unsigned(algorithm.warp_tile_shape.n) << "x"
        << unsigned(algorithm.warp_tile_shape.k);

    if(signature.split_k > 1)
        oss << "_splitk" << unsigned(signature.split_k);
    if(!signature.elementwise_op.empty() && signature.elementwise_op != "PassThrough")
        oss << "_" << signature.elementwise_op;
    if(signature.num_d_tensors > 0)
        oss << "_d" << unsigned(signature.num_d_tensors);
    if(signature.structured_sparsity) oss << "_sparse";
    if(algorithm.preshuffle)         oss << "_preshuffle";
    return oss.str();
}

```

Example output:

```
fp16_rcr_compv3_default_intrawave_False_False_False_False_256x128x32_4x1x1_32x32x16
```

This string is the canonical key. Any mismatch between what codegen writes to the filename and what the runtime computes from a KernelKey causes a silent dispatch failure: `registry.lookup()` returns `nullptr`, the dispatcher falls back to nothing, and every output is wrong with no error message.

Registry — The thread-safe kernel phone book (`registry.hpp`)

```
class Registry : public BaseRegistry<Registry, std::string, KernelInstance> {
public:
    bool                register_kernel(KernelInstancePtr, Priority = Priority::Normal);
    KernelInstancePtr    lookup(const std::string& identifier) const;
    KernelInstancePtr    lookup(const KernelKey& key) const; // calls encode_identifier
    std::vector<KernelInstancePtr> get_all() const;
    std::vector<KernelInstancePtr> filter(...) const;
    std::size_t          filter_by_arch(const std::string& gfx_arch);
    std::string           export_json(bool include_statistics = true) const;
    static Registry&      instance(); // global singleton
};
```

The registry is a `std::map<std::string, KernelInstancePtr>`; protected by a mutex. Lookup is $O(\log N)$ on the identifier string. On initialization, generated kernels self-register via a macro:

```
// In gemm_ctypes_lib.cpp – generated by codegen
REGISTER_GENERATED_KERNELS(*g_registry, GFX_ARCH);
```

This macro is emitted by `generate_dispatcher_registration.py` and expands to one `register_kernel()` call per generated kernel.

KernelInstance — The abstract kernel interface

```
class KernelInstance {
public:
    virtual const KernelKey& get_key() const = 0;
    virtual std::string      get_name() const = 0;
    virtual bool supports(const Problem& problem) const = 0;
    virtual float run(const void* a, const void* b, void* c,
                     const void** d_ptrs, const Problem& problem,
                     void* stream = nullptr) const = 0;
    virtual bool validate(const void* a, const void* b, const void* c,
                        const void** d_ptrs, const Problem& problem,
                        float tolerance = 1e-3f) const = 0;
};
```

Generated concrete classes look like:

```
// Auto-generated by unified_gemm_codegen.py
class Kernel_fp16_rcr_compv3_intrawave_256x128x32_4x1x1_32x32x16 : public KernelInstance {
    using GemmKernel = ck_tile::GemmKernel<GemmPipeline<...>, GemmEpilogue<...>>;
public:
    const KernelKey& get_key() const override { return key_; }
```

```

bool supports(const Problem& p) const override {
    return p.dtype_a == DataType::FP16 &&
           p.M % 256 == 0 && p.N % 128 == 0; // if pad_m=false
}
float run(const void* a, const void* b, void* c,
          const void** d, const Problem& p, void* stream) const override {
    // launch HIP kernel
    GemmKernel::launch(a, b, c, p.M, p.N, p.K, stream);
    return measure_time();
}
};

```

Dispatcher — The orchestrator (dispatcher.hpp)

```

class Dispatcher {
    Registry* registry_;
    HeuristicFunction heuristic_; // optional ML-guided selection
    SelectionStrategy strategy_; // FirstFit or Heuristic

public:
    KernelInstancePtr select_kernel(const Problem& problem) const;
    float run(const void* a, const void* b, void* c,
              const Problem& problem, void* stream = nullptr) const;
    float run_explicit(const std::string& kernel_id, ...) const;
    float run_fused(const void* a, const void* b, void* c,
                    const void** d_ptrs, const Problem& p, void* stream) const;
};

```

`run()` selects the first kernel in the registry whose `supports(problem)` returns true, or uses the heuristic if one is set. `run_explicit()` looks up by identifier string — this is what you use when you want a specific kernel.

The Python interface — `ctypes_utils.py`

The Python side of the Dispatcher is in `dispatcher/python/ctypes_utils.py`. The key classes:

```

class KernelConfig:
    """Describes a kernel to compile/use."""
    tile_m: int; tile_n: int; tile_k: int
    pipeline: str      # "compv3", "compv4", "preshufflev2", ...
    scheduler: str     # "intrawave", "interwave", "auto"
    epilogue: str      # "cshuffle", "default", "none"
    dtype_a: str       # "fp16", "bf16", "fp8", "int8", ...
    pad_m: bool; pad_n: bool; pad_k: bool
    arch: str          # "gfx942", "gfx90a", ...

class Registry:
    """Python-side registry: builds and holds kernels."""
    def build(self, config: KernelConfig) -> None:
        # 1. Calls unified_gemm_codegen.py to generate gemm*.hpp
        # 2. Calls hipcc to compile → libdispatcher_gemm_<hash>.so
        # 3. Loads the .so via ctypes
        # 4. Calls dispatcher_initialize() to register kernel

```

Usage from Python examples (examples/gemm/python/01_basic_gemm.py):

This is the production interface. `reg.build()` handles codegen, compilation, and loading internally. The caller never touches `.hpp` files, never calls codegen scripts directly, never knows about kernel identifiers.

The Dispatcher's supported kernel matrix

dtype	layouts	pipelines	schedulers	GPU targets
fp16, bf16, fp8, bf8, int8, fp32, fp4, crr	mem, compv3, compv4, compv5, shuffle2	auto	gtx908, gtx90a, gtx942, gtx950, gtx1100, gtx1100a	

The Dispatcher already covers everything Tile Engine covered, plus more.

Chapter 4: The Critical Naming Traps

Trap 1: Warp vs Wave — the silent wrong-kernel bug

The GPU hardware executes 64 threads together as a **wavefront** (AMD's term). NVIDIA calls the same concept a **warp**. The CK codebase uses both terms, but not consistently.

In a GEMM tile hierarchy:

The naming inconsistency:

Parameter	What Tile Engine calls it	What Dispatcher calls it	Actual meaning
Wave count per block in M	<code>`warp_m`</code>	<code>`wave_shape.m`</code>	How many wavefronts tile the block in M
Wave count per block in N	<code>`warp_n`</code>	<code>`wave_shape.n`</code>	How many wavefronts tile the block in N
MFMA shape in M	<code>`warp_tile_m`</code>	<code>`warp_tile_shape.m`</code>	Shape of one MFMA instruction

If you swap `warp_m` (e.g. 4) with `warp_tile_m` (e.g. 32), you get a config where:

- The block tile is described correctly
- But the internal wave layout is wrong

The kernel compiles. It launches. It produces wrong answers. There is no error. This is documented in `te_to_dispatcher.py:_Tile` with a warning comment:

```
@dataclass(frozen=True)
class _Tile:
    # NOTE: Naming trap – TE uses "warp_m/n/k" to mean wave counts per block.
    # The dispatcher calls these same values "wave_shape.m/n/k".
    # What the dispatcher calls "warp_tile" is the per-warp MFMA shape.
    # Mixing them produces valid-looking but wrong kernels.
    warp_m: int      # wave COUNT per block (not the MFMA shape)
    warp_tile_m: int # MFMA shape per wave (not the wave count)
```

Trap 2: The dual-name system — two strings, one kernel

Every kernel has **two different name strings** that must never be confused:

Registry identifier — used by the C++ runtime to look up kernels in the registry.

Built from canonical Dispatcher string forms (scheduler "default" → "auto"):

```
fp16_rcr_compv3_default_auto_False_False_False_False_256x128x32_4x1x1_32x32x16
          ^^^^^
          scheduler in canonical dispatcher form
```

TE kernel name — used to find the `.hpp` file on disk and to find the TE benchmark binary. Built from raw TE string forms:

```
fp16_rcr_compv3_default_default_False_False_False_False_256x128x32_4x1x1_32x32x16
          ^^^^^^^^^
          scheduler in raw TE form ("default", not "auto")
```

What breaks if you swap them:

- Use TE name as registry identifier: runtime lookup returns null, dispatcher silently falls back, all outputs are wrong, zero error message
- Use registry identifier to find `.hpp` file: file not found, build fails loudly

The `_preshuffle` suffix makes this worse: `unified_gemm_codegen.py` appends `_preshuffle` to the filename for `preshufflev2` pipeline, but the registry identifier has `_preshuffle` in a different position (as an optional suffix at the end). Both `te_kernel_name()` in `check_parity.py` and `_kernel_name()` in `sweep_runner.py` must mirror the codegen naming exactly.

Trap 3: Scheduler "default" vs "auto"

Tile Engine uses the string "default" to mean "let the system choose a scheduler".

The Dispatcher's C++ enum is `Scheduler::Auto`, whose `to_string()` returns "auto".

```
// kernel_key.hpp
inline std::string to_string(Scheduler scheduler) {
    switch(scheduler) {
        case Scheduler::Auto:      return "auto";      // NOT "default"
        case Scheduler::Intrawave: return "intrawave";
        case Scheduler::Interwave: return "interwave";
    }
}
```

The mapping table in `te_to_dispatcher.py`:

```
_SCHEDULER_CANON = {
    "intrawave": "intrawave",
    "interwave": "interwave",
    "default":   "auto",    # ← TE "default" → Dispatcher "auto"
    "auto":      "auto",
}
```

A TE config with `"scheduler": "default"` produces a Dispatcher identifier with "auto" in the scheduler position, but a TE filename/binary with "default" in that position. These are different strings. Never confuse them.

Chapter 5: The Wrong PR — Detailed Failure Analysis

What the PR built

The current PR (muozturk/dispatcher-te-parity) contains these key files in parity/:

File	Purpose	Problem
<code>`te_to_dispatcher.py`</code>	Translate TE JSON → dispatcher config dict	Preserves TE as source of truth
<code>`identifier.py`</code>	Python reimplementaion of <code>`encode_identifier`</code>	Proves Python/C++ agree — not wrong, just not the right test
<code>`drive_codegen.py`</code>	Invoke <code>`unified_gemm_codegen.py`</code> as subprocess	Treats dispatcher's own tool as external TE tool

File	Purpose	Problem
`harness.cpp`	Custom single-kernel C++ runner	Not the production interface; proves wrong thing
`check_parity.py`	3-stage orchestrator	Stages 2-3 test harness.cpp, not DispatcherLib
`sweep_runner.py`	Batch (kernel, problem) runner	Uses custom harness, not DispatcherLib

The data flow the PR proves

```
TE JSON config
  ──┬─
  │ te_to_dispatcher.py::translate()
  │   ──┬─ (converts TE vocab to dispatcher vocab)
  │   │   ▼
  │   │ dispatcher config dict
  │   │   ──┬─
  │   │   │ drive_codegen.py::drive()
  │   │   │   ──┬─ (invokes unified_gemm_codegen.py as subprocess, builds minimal TE config)
  │   │   │   │   ▼
  │   │   │   │ generated/parity_single/gemm_<te_kernel_name>.hpp
  │   │   │   │   ──┬─
  │   │   │   │   │ build_harness.sh (hipcc harness.cpp + header → ./harness)
  │   │   │   │   │   ──┬─
  │   │   │   │   │   │ ./harness -m=1024 -n=1024 -k=1024 -verify=1
  │   │   │   │   │   │   ──┬─
  │   │   │   │   │   │   │ parse_harness_output() → verdict: PASSED/FAILED, tflops: 84.7
  │   │   │   │   │   │   │   ──┬─
  │   │   │   │   │   │   │   │ compare against:
  │   │   │   │   │   │   │   │ benchmark_gemm_universal_<te_kernel_name> -m=1024 -n=1024 -k=1024 -verify=1
  │   │   │   │   │   │   │   │   ──┬─
  │   │   │   │   │   │   │   │   │ assert: same answer, delta TFLOP/s < 2%
```

What this proves: "my translator + my custom harness agree with TE".

The data flow the PR should prove

```
DispatcherLib.load()      # loads pre-built libdispatcher_gemm_lib.so
  ──┬─
  │ reg = Registry()
  │ cfg = KernelConfig(tile_m=256, tile_n=128, tile_k=32, dtype_a="fp16", ...)
  │ reg.build(cfg)         # JIT: codegen + hipcc inside DispatcherLib
  │   ──┬─
  │   │ result_disp = reg.run(A, B, C, M=1024, N=1024, K=1024)
  │   │   ──┬─
  │   │   │ compare against:
  │   │   │ benchmark_gemm_universal_<name> -m=1024 -n=1024 -k=1024 -verify=1
  │   │   │   ──┬─
  │   │   │   │ assert: same answer, delta TFLOP/s < 2%
```

What this proves: "DispatcherLib — the actual production replacement — agrees with TE".

The three specific mistakes

Mistake 1: The translator makes the Dispatcher speak TE's language.

`te_to_dispatcher.py` accepts a TE JSON config as input and converts it to Dispatcher format. This is architecturally inverted. The Confluence page says: make TE conform to the Dispatcher model, not the other way around. A translator that accepts TE JSON as its primary input format permanently enshrines TE's vocabulary as the entry point to the system — exactly what was asked not to do.

The Dispatcher has its own config format (`KernelConfig` in `ctypes_utils.py`) and its own examples showing how to build configs. The right approach is for callers to learn the Dispatcher's API, not to have a shim that accepts TE configs.

Mistake 2: `drive_codegen.py` treats the Dispatcher's own codegen as an external tool.

`unified_gemm_codegen.py` lives at `dispatcher/codegen/` and is invoked by the Dispatcher's CMake build. It is the Dispatcher's tool. The PR's `drive_codegen.py` calls it as a subprocess from outside the build system to produce a one-off header:

```
cmd = [sys.executable, str(_CODEGEN),
      "--output-dir", str(output_dir),
      "--datatype", te["datatype"],
      "--config", tmp.name, ...]
proc = subprocess.run(cmd, ...)
```

This bypasses CMake entirely. The produced header is not part of the Dispatcher's build artifact. It is an ad-hoc compilation that exists only for the custom harness.

The Dispatcher's production `.so` is never touched.

Mistake 3: `harness.cpp` is not the production interface.

`harness.cpp` is a ~200-line custom C++ program. It allocates buffers, initializes them with a fixed pattern (`h_a[i] = (i%7-3)*0.25`), calls `SelectedKernel::launch`, measures time, and checks against a CPU reference. This is a reasonable integration test for a single kernel, but it is not `DispatcherLib`. Proving `harness.cpp` passes does not prove `DispatcherLib` works.

The entire production replacement path — Registry, Dispatcher, ctypes bridge, Python `KernelConfig`, GPU arch detection, kernel registration macro — is untested by the current PR. When the TE binaries are deleted and callers switch to `DispatcherLib.run_gemm()`, that path could have bugs that the 220 passing tests would never catch.

What the 220 tests actually test

The test suite is in `test_te_to_dispatcher.py` (60 tests) and `test_sweep_and_report.py` (160 tests). They test:

- `translate()` correctly converts TE JSON to dispatcher config dicts
- `encode_identifier()` produces correct strings
- Rejection manifests are generated correctly
- Parquet output schema is correct
- Compare report renders correctly

None of these tests exercise `DispatcherLib`, `Registry.build()`, `Registry.run()`, or the ctypes binding. They are unit tests of the translator and reporting layer — which is not what needs to be proven.

Chapter 6: The Task Description — Multiple Perspectives

Perspective 1: The system architect's view

The task is a **consolidation migration**. Two independently operating systems (TE + Dispatcher) converge into one (Dispatcher only). The migration has three phases:

```
Phase 1: Verify the new system can handle all workloads the old system handled
Metric: 100% of TE CI workloads pass through Dispatcher with ≤2% perf delta

Phase 2: Cut over all consumers from old system to new system
Action: Update every call site from benchmark_gemm_universal_* to DispatcherLib

Phase 3: Decommission the old system
Action: Remove TE codegen path, TE benchmark build targets, TE JSON configs
```

Perspective 2: The software engineer's view

The task is a **replace-and-verify** operation:

```
FIND:    grep -r "benchmark_gemm_universal" . → list of call sites
BUILD:   cmake build the Dispatcher → libdispatcher_gemm_lib.so
VERIFY:  for each (kernel, M, N, K) in CI matrix:
          dispatcher_result = DispatcherLib.run_gemm(...)
          te_result         = subprocess(benchmark_gemm_universal...)
          assert numerical_match(dispatcher_result.C, te_result.C)
          assert abs(dispatcher_result.tflops - te_result.tflops) / te_result.tflops < 0.02
REPLACE: update call sites to use DispatcherLib
DELETE:  remove TE build targets, JSON configs, TE-specific code
```

Perspective 3: Vidya's architectural principle

The key sentence from the Confluence page: *****Do NOT make the dispatcher look like**

Tile Engine. Make Tile Engine conform to the dispatcher model."**

This is an **asymmetric migration rule**. It means:

- The Dispatcher's API, naming conventions, and config format are fixed
- Callers must adapt to the Dispatcher, not the other way around
- No compatibility shims that accept TE-flavored inputs

In practice: if a CI script currently passes a TE JSON config to kick off a GEMM benchmark, that script must be updated to use `KernelConfig` and `Registry.run()`. It is not acceptable to build a layer that accepts TE JSON and internally converts it.

Perspective 4: What "parity" actually means in this context

The word "parity" appears in the Confluence page in the phrase "achieving feature parity." This is not about numerical closeness between TE and Dispatcher outputs for the same input. It is about **feature coverage**:

- Every dtype/layout/pipeline combination that TE CI exercised must be exercisable through the Dispatcher
- Every GPU architecture that TE supported must be supported in the Dispatcher
- Every element-wise fusion variant (PassThrough, MultiDAdd, etc.) must work

Numerical and performance parity (same answers, same TFLOP/s) are the *evidence* that feature parity has been achieved correctly — not the definition of the goal.

Perspective 5: What "deletion" really means

The Confluence page says TE should be deleted and replaced by a thin Python/JIT frontend. The "thin frontend" is not a new thing to build — it already exists:

```
# This IS the thin frontend
from ctypes_utils import KernelConfig, Registry
reg = Registry()
reg.build(KernelConfig(tile_m=256, ...))
result = reg.run(A, B, C, M, N, K)
```

"Deletion" means:

- Remove the TE benchmark binary build targets from CMake
- Remove the TE-specific JSON configs that drove those builds
- Remove any code that invokes `benchmark_gemm_universal_*` directly
- Keep: `dispatcher/codegen/unified_gemm_codegen.py` (Dispatcher's tool, not TE's)
- Keep: `dispatcher/python/ctypes_utils.py` (the thin frontend)
- Keep: `dispatcher/include/`, `dispatcher/src/` (C++ core)

Chapter 7: The Correct Implementation Plan

Step 0: Understand the current TE CI footprint

Before writing code, answer these questions with `grep` and `git log`:

```
# Find all TE binary invocations
grep -r "benchmark_gemm_universal" . --include="*.py" --include="*.sh" -l

# Find all TE JSON config files
find . -name "*.json" | xargs grep -l "tile_config" 2>/dev/null

# Find CMake targets that build TE binaries
grep -r "benchmark_gemm_universal" . --include="CMakeLists.txt"
```

The output defines the **exact scope** of the migration. If there are 3 call sites, the work is small. If there are 50, it is larger. You cannot scope the work without this.

Step 1: Build the Dispatcher and inventory its kernels

```
cd dispatcher
cmake -B build -DCMAKE_BUILD_TYPE=Release -DGPU_TARGETS="gfx942"
cmake --build build --parallel $(nproc)
```

Then in Python:

```
from ctypes_utils import Registry, KernelConfig, detect_gpu_arch

arch = detect_gpu_arch()
reg = Registry()

# Build a representative kernel and list what's registered
cfg = KernelConfig(tile_m=256, tile_n=128, tile_k=32,
                    pipeline="compv3", scheduler="intrawave",
                    dtype_a="fp16", arch=arch)
reg.build(cfg)

for kernel in reg.list_kernels():
    print(kernel.identifier)
```

Cross-reference this list against the TE configs from Step 0. Any TE config not covered by the Dispatcher is a gap that must be fixed in the Dispatcher's build config before you can proceed.

Step 2: Write the parity proof through DispatcherLib

```
import subprocess, numpy as np
from ctypes_utils import Registry, KernelConfig, detect_gpu_arch

def run_dispatcher(dtype, tile_m, tile_n, tile_k, pipeline, M, N, K):
```

```

arch = detect_gpu_arch()
reg = Registry()
cfg = KernelConfig(tile_m=tile_m, tile_n=tile_n, tile_k=tile_k,
                    pipeline=pipeline, scheduler="intrawave",
                    dtype_a=dtype, arch=arch)

reg.build(cfg)
A = np.random.randn(M, K).astype(np.float16)
B = np.random.randn(K, N).astype(np.float16)
C = np.zeros((M, N), dtype=np.float16)
return reg.run(A, B, C, M=M, N=N, K=K) # returns GemmResult

def run_te(te_binary_name, M, N, K):
    proc = subprocess.run(
        [f"./{te_binary_name}", f"-m={M}", f"-n={N}", f"-k={K}", "-verify=1"],
        capture_output=True, text=True)
    # parse TFLOP/s from output
    ...

# For each (kernel, problem size) in CI matrix:
for kernel_spec, (M, N, K) in CI_MATRIX:
    disp = run_dispatcher(**kernel_spec, M=M, N=N, K=K)
    te = run_te(kernel_spec["te_binary_name"], M, N, K)
    assert disp.verdict == "PASSED"
    assert te.verdict == "PASSED"
    delta = abs(disp.tflops - te.tflops) / te.tflops
    assert delta < 0.02, f"PERF REGRESSION: {delta:.1%} > 2%"

```

This is the proof that DispatcherLib is safe to use as a replacement. It tests the actual production code path, not a custom harness.

Step 3: Write the thin frontend

```

# gemm_runner.py – drop-in replacement for benchmark_gemm_universal_*
"""
Thin frontend: run a GEMM via the Dispatcher.
Replaces: ./benchmark_gemm_universal_<name> -m=M -n=N -k=K -verify=1
Usage:    python gemm_runner.py --dtype fp16 --tile 256x128x32
          --pipeline compv3 --m 1024 --n 1024 --k 1024
"""

import argparse
from ctypes_utils import Registry, KernelConfig, detect_gpu_arch

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--dtype", default="fp16")
    p.add_argument("--tile", default="256x128x32") # tile_m x tile_n x tile_k
    p.add_argument("--pipeline", default="compv3")
    p.add_argument("--scheduler", default="intrawave")
    p.add_argument("--m", type=int, required=True)
    p.add_argument("--n", type=int, required=True)
    p.add_argument("--k", type=int, required=True)
    p.add_argument("--verify", action="store_true")
    args = p.parse_args()

    tile_m, tile_n, tile_k = map(int, args.tile.split("x"))
    arch = detect_gpu_arch()
    reg = Registry()

```

```

    cfg = KernelConfig(tile_m=tile_m, tile_n=tile_n, tile_k=tile_k,
                       pipeline=args.pipeline, scheduler=args.scheduler,
                       dtype_a=args.dtype, arch=arch)
    reg.build(cfg)

    result = reg.run_benchmark(M=args.m, N=args.n, K=args.k, verify=args.verify)
    print(f"{'PASSED' if result.correct else 'FAILED'}")
    print(f"Time: {result.time_ms:.2f} ms  ({result.tflops:.1f} GFLOP/s)")
    return 0 if result.correct else 1

if __name__ == "__main__":
    raise SystemExit(main())

```

This is thin: it does not know kernel internals. It speaks the Dispatcher's vocabulary (KernelConfig), not TE's vocabulary (JSON configs). Callers update their scripts to call this instead of `benchmark_gemm_universal_*`.

Step 4: Update call sites

Every place found in Step 0 that calls `benchmark_gemm_universal_<name>` becomes:

```

# Before
./benchmark_gemm_universal_fp16_rcr_compv3_..._256x128x32_4x1x1_32x32x16 \
-m=1024 -n=1024 -k=1024 -verify=1

# After
python gemm_runner.py --dtype fp16 --tile 256x128x32 \
--pipeline compv3 --m 1024 --n 1024 --k 1024 --verify

```

Step 5: Delete TE

Remove from CMake:

```

# DELETE these targets:
add_executable(benchmark_gemm_universal_fp16_rcr_compv3_... ...)
# etc.

```

Remove from the repository:

- TE-specific JSON config files used to drive TE binary builds
- TE build scripts
- Any `harness.cpp` or `drive_codegen.py` that only served the custom harness path

Keep everything in dispatcher/:

- `codegen/unified_gemm_codegen.py` — Dispatcher's own codegen, not TE
- `python/ctypes_utils.py` — the thin frontend base
- `include/`, `src/`, `bindings/` — Dispatcher C++ core
- `examples/gemm/python/` — usage examples

Chapter 8: What to Do with the Current PR Code

Keep (after repurposing)

`identifier.py` **and** `check_identifier_parity.py` — The proof that Python and C++ `encode_identifier()` agree byte-for-byte is genuinely useful as a regression test. Keep these. They prevent the silent dispatch failure caused by identifier mismatch.

The test structure — The test framework (pytest, conftest, parametrize patterns) is good. Repurpose the tests to cover the right things (DispatcherLib behavior, not the translator).

Discard

`te_to_dispatcher.py` — The translator. Its entire purpose was to keep TE JSON as the input format. Under the new direction, callers use `KernelConfig` directly.

`drive_codegen.py` — The codegen subprocess invoker. `Registry.build()` in `ctypes_utils.py` already does this correctly within the Dispatcher's own machinery.

`harness.cpp` **and** `build_harness.sh` — The custom single-kernel runner. Replaced by `Registry.run()` from `ctypes_utils.py`.

`check_parity.py` — The 3-stage orchestrator. Its Stage 2 and 3 test the custom harness. Replaced by the new parity proof that uses `DispatcherLib` directly.

`sweep_runner.py` — The batch runner. Uses the custom harness. Replaced by a version that uses `Registry.run()`.

Rewrite

The parity runner — New version: takes a list of `KernelConfig` specs and problem sizes, runs each through `Registry.build() + Registry.run()`, records results in Parquet. No subprocess codegen, no custom harness.

The compare report — Keep the reporting logic. Update the data source to come from the new parity runner.

The test suite — Rewrite to test: `Registry.build()` produces a loadable `.so`, `Registry.run()` returns correct answers for small known inputs, thin frontend CLI produces expected output format.

Chapter 9: Summary of All Key Points

Architecture summary

```
unified_gemm_codegen.py
  Owner: Dispatcher (lives in dispatcher/codegen/)
  Role: Generate C++ kernel headers from config
  Called by: CMake (build time) AND Registry.build() (JIT)
  NOT a TE tool – TE was using the Dispatcher's tool

KernelKey / encode_identifier()
  The single source of truth for kernel identity
  Python (identifier.py) and C++ (kernel_key.hpp) must agree byte-for-byte
  Mismatch = silent dispatch failure, wrong outputs, no error message

Registry
  Thread-safe map: identifier string → KernelInstance
  Populated at library load time by REGISTER_GENERATED_KERNELS macro
  Can also be populated JIT via Registry.build() in Python

DispatcherLib / Registry (Python)
  The production replacement for benchmark_gemm_universal_*
  Lives in dispatcher/python/ctypes_utils.py
  Handles: arch detection, JIT codegen, compilation, ctypes loading, dispatch

Thin frontend (to be built)
  A small Python script wrapping DispatcherLib
  Accepts the same problem description a TE caller would provide
  In Dispatcher vocabulary, not TE vocabulary
```

The one-page decision tree

```
Q: What system should callers use?
A: DispatcherLib (dispatcher/python/ctypes_utils.py)

Q: How is DispatcherLib loaded?
A: DispatcherLib.load() or Registry() → reg.build(KernelConfig(...))

Q: How are kernels generated?
A: By unified_gemm_codegen.py (called internally by CMake or Registry.build())

Q: What is the kernel identity string?
A: KernelKey::encode_identifier() – Python and C++ must agree

Q: What replaces benchmark_gemm_universal_*?
A: gemm_runner.py (thin frontend) calling Registry.run()

Q: What gets deleted?
A: TE benchmark build targets, TE JSON configs, te_to_dispatcher.py,
  drive_codegen.py, harness.cpp

Q: What stays?
A: dispatcher/ everything – it is the Dispatcher's code, not TE's
```

Why this is the right direction

The Dispatcher was built to replace Tile Engine. It has a complete Python interface, a production-grade C++ registry, JIT compilation support, and multi-kernel dispatch. All of this existed before the parity PR was written. The parity PR built parallel infrastructure that tests a different path. The correct PR tests and then enables the path that was already built, then removes the path it replaced.